Crash Course in Maths for Cognitive Science

Session 4 | Linear Algebra II

Ali Shiravand

ali.shiravand@ens.psl.eu

September 2026

Session 4 | Linear Algebra II

What today is about: the payoff session

Today the two halves of the course meet.

- *Eigenvectors* and *eigenvalues*: the special directions of a transformation.
- **Principal Component Analysis**: the flagship tool for reducing and visualising high-dimensional data.
- The *SVD*: the general engine behind PCA.
- *Dynamics*: how eigenvalues decide whether a system settles, oscillates, or explodes.

We build slowly, with a picture and a worked example at every step.

Where statistics and linear algebra become one

Key idea

The **covariance matrix** from Session 2 (a statistical object) read through its **eigenvectors** (a linear-algebra tool) *is* Principal Component Analysis. That single sentence is the destination of today.

So keep the covariance matrix in mind as we build the eigenvalue machinery; we are assembling exactly the pieces PCA needs.

Recall: a matrix moves vectors around

From last session: multiplying a vector by a matrix $\boldsymbol{A}$ *transforms* it, it usually lands pointing in a new direction.

Today's question is simple and powerful:

*Are there any special directions that the matrix does **not** turn?*

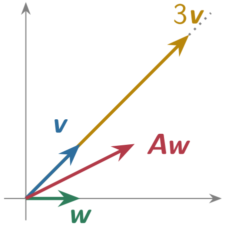
The answer, yes, a few, unlocks PCA and the behaviour of dynamical systems. Let us just *look* at what a matrix does to a handful of vectors.

Watch what a matrix does to a few vectors

Take $\boldsymbol{A} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ and apply it:

- $\boldsymbol{A}\left(\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right)$, turned off its line.
- $\boldsymbol{A}\left(\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 3 \\ 3 \end{smallmatrix}\right) = 3\left(\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right)$, stayed on its line, just $3\times$ longer!

The direction $(1, 1)$ is *special*: $\boldsymbol{A}$ only stretches it.



The special directions have a name

Definition

A nonzero vector $\boldsymbol{v}$ is an **eigenvector** of $\boldsymbol{A}$ if $\boldsymbol{A}$ only *stretches* it, without changing its direction:

$$\boldsymbol{A}\boldsymbol{v} = \lambda\,\boldsymbol{v},$$

where the number λ is the **eigenvalue**, the stretch factor.

In the example, $(1, 1)$ is an eigenvector with eigenvalue $\lambda = 3$. ("Eigen" is German for "own": these are the matrix's *own* directions.)

Eigenvectors are the “grain of the wood”

Intuition

Think of the transformation as flowing space around. Almost every arrow gets rotated as well as stretched. But along an eigenvector, the transformation is pure stretch: the arrow stays on its own line, only getting longer, shorter, or flipped.

Along these special directions a complicated matrix behaves like plain multiplication by a number. That simplicity is what makes eigenvectors so powerful.

How do we *find* the special directions?

We want $\boldsymbol{Av} = \lambda \boldsymbol{v}$ for some nonzero $\boldsymbol{v}$. Rearrange:

$$\boldsymbol{Av} - \lambda \boldsymbol{v} = 0 \quad \Longrightarrow \quad (\boldsymbol{A} - \lambda \boldsymbol{I})\, \boldsymbol{v} = 0.$$

Intuition

A matrix that sends a *nonzero* vector to 0 must **collapse space**, and from last session, that means its determinant is zero. So the special stretch factors λ are exactly the ones with

$$\boxed{\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0} \qquad \text{(the \textit{characteristic equation})}.$$

Finding eigenvalues: try it

Example

For $\boldsymbol{A} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, find the eigenvalues by solving $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$, then find an eigenvector for each.

Try it yourself first — the solution is on the next slide.

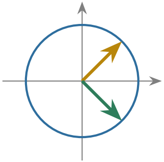
Finding eigenvalues: solution

$$\det\begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = 0 \implies 2-\lambda = \pm 1 \implies \lambda = 3 \text{ or } 1.$$

The stretch factors are 3 and 1. Their directions come from $(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{v} = 0$: they are $(1, 1)$ for $\lambda = 3$ and $(1, -1)$ for $\lambda = 1$, and they are *orthogonal*, as they must be for a symmetric matrix.

Seeing the eigen-directions: circle → ellipse

A symmetric matrix sends the unit circle to an *ellipse*. Its axes point along the eigenvectors, and their half-lengths are the eigenvalues.



Unit circle, with the two eigen-directions $(1, 1)$ and $(1, -1)$.

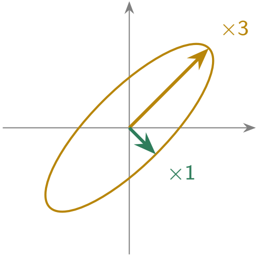


Image ellipse: stretched $\times 3$ along $(1, 1)$, $\times 1$ along $(1, -1)$.

This ellipse *is* the picture of PCA: the long axis is the first principal component, and λ is the variance along it.

Two free checks on your eigenvalues

Useful fact

For any square matrix:

$$\sum_i \lambda_i = \operatorname{tr}(\boldsymbol{A}) \text{ (sum of the diagonal)}, \qquad \prod_i \lambda_i = \det(\boldsymbol{A}).$$

For the example above: $\lambda_1 + \lambda_2 = 3 + 1 = 4 = \operatorname{tr}(\boldsymbol{A})$, and $\lambda_1\lambda_2 = 3 \cdot 1 = 3 = \det(\boldsymbol{A})$.
Always use these to catch arithmetic slips before going any further.

Eigenvalues you can read off without any work

Useful fact

- A *diagonal* or *triangular* matrix has its eigenvalues sitting on the diagonal.
- If $\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v}$, then (same eigenvector $\boldsymbol{v}$): $\boldsymbol{A}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$, $\boldsymbol{A}^{-1}\boldsymbol{v} = \frac{1}{\lambda}\boldsymbol{v}$, and $(\boldsymbol{A} + c\boldsymbol{I})\boldsymbol{v} = (\lambda + c)\boldsymbol{v}$.

Intuition

Eigenvectors are unchanged by powering, inverting, or shifting a matrix, only the eigenvalue is transformed the obvious way. This is why, once you know the eigen-directions, *every* power of the matrix becomes easy.

A non-symmetric example: try it

Example

Find the eigenvalues and eigenvectors of the (non-symmetric) matrix

$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}.$$

Are the eigenvectors orthogonal this time? (Use the trace/det check too.)

Try it yourself first — the solution is on the next slide.

A non-symmetric example: solution

The matrix is triangular, so the diagonal already gives the eigenvalues:
$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = (2-\lambda)(3-\lambda) = 0 \Rightarrow \lambda = 2, 3$. Check: $2 + 3 = 5 = \operatorname{tr} \boldsymbol{A}$, $2 \cdot 3 = 6 = \det \boldsymbol{A}$.
✓ The eigenvectors are $(1, 0)$ for $\lambda = 2$ and $(1, 1)$ for $\lambda = 3$.

Intuition

Here $(1, 0) \cdot (1, 1) = 1 \neq 0$: the eigenvectors are *not* orthogonal. Only **symmetric** matrices are guaranteed perpendicular eigen-directions, and covariance matrices are always symmetric, which is exactly why PCA gives clean, perpendicular axes.

A reflection: try it

Example

The matrix $\boldsymbol{A} = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ reflects the plane across the line $y = x$. Without much algebra, guess which two directions it leaves on their own line, and with what stretch factors. Then confirm with the characteristic equation.

Try it yourself first — the solution is on the next slide.

A reflection: solution

Points *on* the mirror line $y = x$ do not move: direction $(1, 1)$ has $\lambda = 1$. Points *perpendicular* to it get flipped to the other side: direction $(1, -1)$ has $\lambda = -1$. The algebra agrees: $\det\left(\begin{smallmatrix} -\lambda & 1 \\ 1 & -\lambda \end{smallmatrix}\right) = \lambda^2 - 1 = 0 \Rightarrow \lambda = \pm 1$.

Intuition

A *negative* eigenvalue means that direction is **flipped** ($|\lambda| = 1$, so no stretching: a reflection preserves length). Sign of λ: $+$ keeps direction, $-$ reverses it, 0 collapses it.

When a matrix has *no* nice real eigenvectors

Common pitfall

Not every matrix has two independent real eigenvectors.

- A **rotation** $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (quarter turn) leaves *no* real direction fixed, its eigenvalues are *complex*, $\lambda = \pm i$. Complex eigenvalues signal *rotation / oscillation* (we will meet them again in dynamics).
- A **shear** $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has a *repeated* eigenvalue $\lambda = 1$ but only *one* eigen-direction $(1, 0)$, it is called *defective*.

The good news: the matrices we care about most, *symmetric* covariance matrices, never have either problem.

Symmetric matrices: perpendicular axes

Key idea

If $\boldsymbol{A} = \boldsymbol{A}^{\mathsf{T}}$ (symmetric, as *every* covariance matrix is), then its eigenvalues are real and its eigenvectors are mutually *orthogonal* (perpendicular). The transformation is then just independent stretches along a set of perpendicular axes.

This clean structure, perpendicular directions, each with its own stretch, is precisely what makes PCA work.

A friendly special case: positive-definite

Definition

A symmetric matrix is **positive-definite** when all its eigenvalues are *positive*.

Intuition

Positive eigenvalues mean the matrix *stretches outward* along every axis, it never flips a direction or collapses it. Eigenvalues 3 and 1: positive-definite. Eigenvalues 3 and -1: *not* (one direction gets flipped).

Covariance matrices are always positive-semidefinite (eigenvalues ≥ 0), because each eigenvalue is an *amount of variance*, and variance can never be negative. That is why PCA's "variance along a component" is never negative.

Eigenvalues make repeated action easy

Intuition

Along an eigenvector, applying $\boldsymbol{A}$ just multiplies by λ. So applying $\boldsymbol{A}$ *again and again* multiplies by λ each time: after k steps, the component is scaled by λ^k.

Formally: put the eigenvectors as the *columns* of a matrix $\boldsymbol{P}$, and the eigenvalues on the *diagonal* of $\boldsymbol{D}$. Then $\boldsymbol{A} = \boldsymbol{P}\boldsymbol{D}\boldsymbol{P}^{-1}$, and raising to a power touches only the middle:

$$\boldsymbol{A}^k = \boldsymbol{P}\,\boldsymbol{D}^k\,\boldsymbol{P}^{-1}, \qquad \boldsymbol{D}^k = \mathrm{diag}(\lambda_1^k, \lambda_2^k, \dots).$$

Powers via eigenvectors: try it

Example

With $\boldsymbol{A} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ (eigenpairs $\lambda = 3$ along $(1, 1)$, $\lambda = 1$ along $(1, -1)$), compute $\boldsymbol{A}^3\left(\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right)$ *without* multiplying the matrix three times. *Hint:* write $(1, 0)$ in the eigenvector basis first.

Try it yourself first — the solution is on the next slide.

Powers via eigenvectors: solution

Split $(1,0)$ into eigen-components: $(1,0) = \frac{1}{2}(1,1) + \frac{1}{2}(1,-1)$. Each component just scales by λ^3:

$$\boldsymbol{A}^3(1,0) = \tfrac{1}{2}\,3^3(1,1) + \tfrac{1}{2}\,1^3(1,-1) = \tfrac{27}{2}(1,1) + \tfrac{1}{2}(1,-1) = (14,13).$$

Intuition

The $\lambda = 3$ direction has already grown $27\times$ while the $\lambda = 1$ direction stayed put, so after a few steps the state is almost pure $(1,1)$. The biggest eigenvalue takes over. Hold onto this: it is exactly how dynamics behave.

An eigenvector you already rely on: $\lambda = 1$

Link to cognitive science

A **Markov chain** (random walk between states) is run by a transition matrix $\boldsymbol{P}$ whose columns sum to 1. Its *long-run distribution* is the eigenvector with eigenvalue $\lambda = 1$: the distribution that no longer changes, $\pi = \boldsymbol{P}\pi$. This drives models of decision states, web-page ranking (PageRank), and word-transition models of language.

Example

$\boldsymbol{P} = \begin{pmatrix} 0.9 & 0.5 \\ 0.1 & 0.5 \end{pmatrix}$: solving $\pi = \boldsymbol{P}\pi$ gives $\pi = \left(\frac{5}{6}, \frac{1}{6}\right)$, the fraction of time spent in each state in the long run. (The other eigenvalue, 0.4, governs how fast the walk forgets where it started.)

Practice: eigenvalues

Practice — try it

Let $\boldsymbol{A} = \begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix}$ (already diagonal).

1. What are its eigenvalues and eigenvectors?
2. Check the trace and the determinant against your eigenvalues.
3. What does $\boldsymbol{A}$ do to the vector $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$? Is $\boldsymbol{A}$ positive-definite?

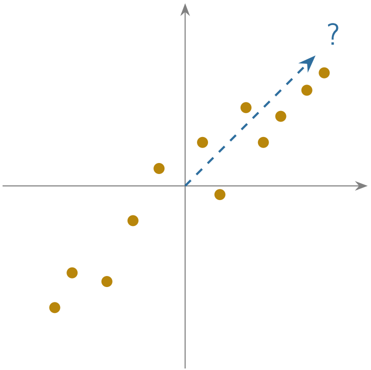
PCA starts with a picture

Suppose two measurements are correlated, so the data form a tilted, cigar-shaped cloud.

Intuition

If you had to keep just *one* number per point, one direction to measure along, which direction loses the least information?

The *long axis* of the cloud. PCA finds that direction automatically.



The problem PCA solves

High-dimensional data are hard to see and often redundant. Because many measured variables are correlated, the cloud of data points really lies close to a lower-dimensional subspace.

PCA finds the few directions that capture the most variation, so we can describe, visualise, and compress the data with little loss. It answers: *along which directions does the data actually vary?*

PCA in one sentence

Key idea

The principal components are the **eigenvectors of the covariance matrix**, and each eigenvalue is the **amount of variance** along that direction. The first component is the direction of greatest spread; the next is the greatest remaining spread orthogonal to it; and so on.

Everything from the first half of today, eigenvectors, symmetric matrices, orthogonality, comes together in this one idea.

Two ways to see the *same* answer

Intuition

PC1 can be defined two equivalent ways, and it is worth holding both:

- the direction of **maximum variance**, spread the projected points out as much as possible;
- the direction of **minimum reconstruction error**, the line the points sit closest to.

Maximising spread *along* a line and minimising distance *to* the line are the same optimisation, which is why PCA is both "the directions of most variance" and "the best-fitting low-dimensional summary".

The PCA recipe

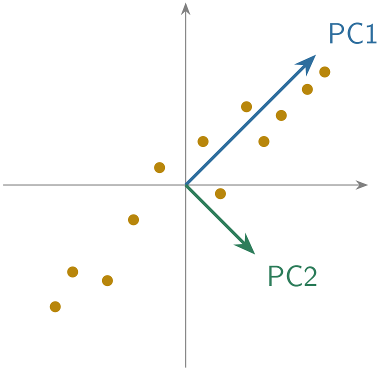
1. **Centre** the data: subtract each variable's mean, so the cloud sits at the origin.
2. Form the **covariance matrix** $\boldsymbol{C}$ of the centred data.
3. Compute the **eigenvectors and eigenvalues** of $\boldsymbol{C}$.
4. Order the eigenvectors by eigenvalue, largest first: these are principal components $1, 2, 3, \ldots$
5. **Project** the data onto the top few components to reduce dimension.

What PCA is doing, geometrically

Intuition

PCA finds the *long axis* of the cloud (most variance, PC1) and the short axis perpendicular to it (PC2). Rotating onto these axes "straightens" the cloud so its directions become uncorrelated.

Keeping only the long axis is dimensionality reduction: describe each point by its position along the direction that matters most.



PCA: try it

Example

Two mean-centred variables have covariance matrix $\boldsymbol{C} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Find the principal components and the fraction of variance the *first* one explains.

Try it yourself first — the solution is on the next slide.

PCA: solution

We already found the eigenpairs of this matrix: $\lambda_1 = 3$ along $(1,1)$, and $\lambda_2 = 1$ along $(1,-1)$. So the first principal component points along $(1,1)$ (the two variables rise together) and carries variance 3; the second carries variance 1. The fraction of variance explained by the first component is

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{3}{4} = 75\%.$$

Keeping only PC1 would summarise each point by one number and retain 75% of the spread.

PCA: a second one to try

Example

Now the covariance matrix is $\boldsymbol{C} = \left(\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right)$. Find its eigenvalues, the first principal component, and the fraction of variance it explains.

Try it yourself first — the solution is on the next slide.

PCA: second solution

Characteristic equation: $(5-\lambda)(2-\lambda)-4=\lambda^2-7\lambda+6=0$, so $\lambda=6$ or 1 (check: $6+1=7=\operatorname{tr}\boldsymbol{C}$, $6\cdot 1=6=\det\boldsymbol{C}$). The PC1 direction solves $(\boldsymbol{C}-6\boldsymbol{I})\boldsymbol{v}=0$, giving $\boldsymbol{v}=(2,1)$; the PC2 direction is $(1,-2)$, orthogonal to it. The first component explains

$$\frac{6}{6+1}=\frac{6}{7}\approx 86\%$$

of the variance, so a single direction already captures most of this cloud.

Projecting a point onto a component: try it

Example

Using the first example (PC1 along $(1, 1)$, PC2 along $(1, -1)$), a data point sits at $\boldsymbol{x} = (3, 1)$. What are its two **PC scores**, its coordinates in the new, rotated axes? *Use unit vectors* $\hat{\boldsymbol{u}}_1 = \frac{1}{\sqrt{2}}(1, 1)$ and $\hat{\boldsymbol{u}}_2 = \frac{1}{\sqrt{2}}(1, -1)$.

Try it yourself first — the solution is on the next slide.

Projecting a point onto a component: solution

A PC score is just the dot product of the point with the (unit) component direction:

$$\text{score}_1 = \hat{\boldsymbol{u}}_1 \cdot \boldsymbol{x} = \frac{3+1}{\sqrt{2}} = \frac{4}{\sqrt{2}} \approx 2.83, \qquad \text{score}_2 = \hat{\boldsymbol{u}}_2 \cdot \boldsymbol{x} = \frac{3-1}{\sqrt{2}} = \frac{2}{\sqrt{2}} \approx 1.41.$$

Intuition

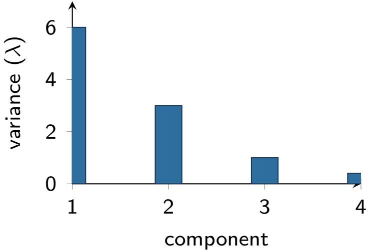
The point $(3, 1)$ becomes (2.83, 1.41) in the rotated PC axes. Dimensionality reduction means keeping only score_1, one number that already places the point along the direction of greatest spread. Projection (Session 3) *is* how PCA compresses.

Reading the eigenvalues

Useful fact

Total variance $= \sum_i \lambda_i = \mathrm{tr}(\boldsymbol{C})$. The proportion explained by component j is $\lambda_j / \sum_i \lambda_i$. All $\lambda_i \geq 0$ (they are variances).

Plot the eigenvalues in decreasing order, the **scree plot**. The "elbow" tells you how many components are worth keeping.



Keep the first two, the rest add little.

A handful of facts that make PCA trustworthy

Useful fact

- The principal components are *orthonormal*: perpendicular, unit length.
- In PC coordinates the variables are *uncorrelated*, PCA diagonalises the covariance matrix.
- The variance of the j-th score is exactly λ_j; total variance is preserved, $\sum_j \lambda_j = \mathrm{tr}(\boldsymbol{C})$.

So PCA is a pure *rotation* of the axes: it never invents or destroys variance, it only re-files it onto perpendicular directions ordered by importance.

Two cautions about PCA

Common pitfall

PCA is a *linear* method and is sensitive to *scaling*. A variable measured in large units (reaction time in milliseconds) can dominate the components purely because of its units, standardise the variables, or use the correlation matrix. And the sign of an eigenvector is *arbitrary*: $\boldsymbol{v}$ and $-\boldsymbol{v}$ are the same component, so never read meaning into the sign alone.

Also remember: PCA finds directions of *variance*, which are not always the directions that matter for a task. Variance is not the same as relevance.

PCA everywhere in cognitive science

Link to cognitive science

PCA reduces recordings of hundreds of neurons to a few "latent" dimensions along which population activity evolves (neural state-space analysis); it compresses fMRI voxels and EEG channels; it extracts the principal components of face images ("eigenfaces"); and it summarises questionnaire batteries into a few underlying factors, the historical root of g in intelligence research and of the "Big Five" in personality.

Whenever someone says a system is "effectively low-dimensional", PCA is usually how they found out.

Practice: PCA

Practice — try it

After PCA on some neural data, the covariance has eigenvalues $\lambda = (6, 3, 1)$ along three orthogonal components.

1. What is the total variance?
2. What fraction does the first component explain?
3. How many components are needed to keep at least 85% of the variance?

The SVD: the general engine behind PCA

Definition

The **Singular Value Decomposition** writes *any* matrix as $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$: a rotation, a set of non-negative stretches (the *singular values* in $\boldsymbol{\Sigma}$), and another rotation.

The idea to carry away: *every* linear transformation is a rotation, then a scaling along perpendicular axes, then another rotation, the circle-to-ellipse picture, now for *any* matrix, not just symmetric ones.

How the SVD relates to PCA

Useful fact

For a centred data matrix $\boldsymbol{X}$, the singular values are the square roots of the covariance eigenvalues: $\sigma_i = \sqrt{\lambda_i}$, and the right singular vectors $\boldsymbol{V}$ *are* the principal components. The number of nonzero singular values is the *rank*, the true dimensionality of the data.

Link to cognitive science

The same decomposition powers *latent semantic analysis* (finding topics in word–document matrices) and *recommender systems* (filling in a users×items matrix from a few latent factors). PCA, LSA, and collaborative filtering are one algorithm wearing three hats.

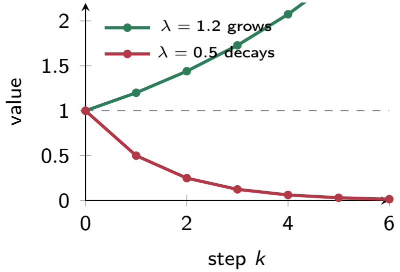
Eigenvalues as growth rates: the one-number version

Start with a single number and multiply it by the same factor over and over:

- factor 0.5: $1 \to 0.5 \to 0.25 \to \cdots \to 0$ (**decays**).
- factor 1.2: $1 \to 1.2 \to 1.44 \to \cdots \to \infty$ (**grows**).

Intuition

An eigenvalue is exactly this repeated factor, but along an eigenvector direction of a matrix. So $|\lambda| < 1$ means that direction dies away; $|\lambda| > 1$ means it blows up.



Repeated multiplication: below 1 dies, above 1 explodes.

Eigenvalues predict the future of a system

Many models describe how a state $\boldsymbol{x}$ evolves, step by step or continuously.

Key idea

Discrete time $\boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t$: stable (shrinks to zero) if every $|\lambda| < 1$; blows up if some $|\lambda| > 1$.

Continuous time $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$: stable if every eigenvalue has *negative real part*; imaginary parts produce oscillations.

Why eigenvalues govern stability

Intuition

Decompose any starting state into its eigenvector components; each then evolves independently, growing or decaying at its own rate, exactly the λ^k we saw when powering a matrix. The largest eigenvalue eventually dominates and sets the long-run behaviour.

This is why the eigenvalues alone tell you whether a dynamical system settles, oscillates, or explodes, without solving anything in detail.

Stability at a glance

Useful fact

	Discrete $\boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t$	Continuous $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$
stable (decays)	all $\lvert\lambda\rvert < 1$	all $\operatorname{Re}\lambda < 0$
marginal	largest $\lvert\lambda\rvert = 1$	largest $\operatorname{Re}\lambda = 0$
unstable (grows)	some $\lvert\lambda\rvert > 1$	some $\operatorname{Re}\lambda > 0$
oscillation	complex λ	nonzero $\operatorname{Im}\lambda$

The marginal case (λ right on the boundary) is where a system neither grows nor decays, the mathematical home of persistent memory, as we will see.

Dynamics: try it

Example

A discrete system $\boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t$ has eigenvalues $\lambda_1 = 0.5$ and $\lambda_2 = 1.2$. What happens to a typical starting state over time, stable or not?

Try it yourself first — the solution is on the next slide.

Dynamics: solution

Along the first eigenvector the state is multiplied by 0.5 each step, so that component decays to zero. Along the second it is multiplied by 1.2, growing by 20% each step. Because one $|\lambda| > 1$, the system is **unstable**: almost every starting state eventually grows without bound. It would be stable only if *both* eigenvalues had magnitude below 1.

Intuition

The largest $|\lambda|$ wins in the long run: it sets the eventual growth or decay rate, and its eigenvector sets the direction the state lines up with.

A system that spirals: try it

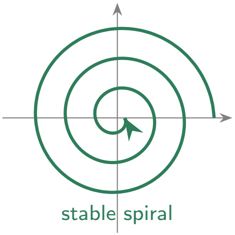
Example

A continuous system $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ has $\boldsymbol{A} = \left(\begin{smallmatrix} -1 & -2 \\ 2 & -1 \end{smallmatrix}\right)$, whose eigenvalues are $\lambda = -1 \pm 2i$. Does it settle, oscillate, or blow up?

Try it yourself first — the solution is on the next slide.

A system that spirals: solution

The eigenvalues are complex, so the state *rotates* (imaginary part ± 2, the oscillation frequency). Their real part is $-1 < 0$, so the amplitude *decays*. The result is a **stable spiral**: the state circles the origin while shrinking towards it, a damped oscillation.



Real part sets growth/decay; imaginary part sets the wobble. Both live in the eigenvalues.

Dynamics in the brain

Link to cognitive science

Recurrent networks and models of neural population dynamics are exactly systems $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$ (plus inputs and nonlinearities). The eigenvalues of the connectivity matrix decide the behaviour:

- all $\operatorname{Re}\lambda < 0$: activity *decays*, a transient response.
- an eigenvalue at $\lambda \approx 0$ (marginal): activity *persists*, a **line attractor**, the standard model of working memory and of neural integrators (e.g. the oculomotor integrator).
- complex eigenvalues: *oscillations*, rhythms, central pattern generators.

Whether a memory holds, a pattern fades, or a network rings is read straight off the eigenvalues, closing the loop with the differential-equations material elsewhere in the programme.

Practice: dynamics

Practice — try it

Stable or unstable?

1. A continuous system with eigenvalues -1 and -3.
2. A continuous system with eigenvalues -2 and $+0.5$.
3. A discrete system with eigenvalues 0.9 and 0.4.
4. A continuous system with eigenvalues $-0.5 \pm 3i$, and does it oscillate?

Session 4: what we built

- *Eigenvectors* are the directions a matrix only stretches, by factor λ; symmetric matrices have real eigenvalues and orthogonal eigenvectors, and powers act through λ^k.
- *PCA* uses the eigenvectors of the covariance matrix as principal components, and the eigenvalues as the variances along them; projecting onto them compresses the data.
- The *SVD* is the general rotation–scale–rotation behind PCA, LSA, and recommender systems.
- In *dynamics*, eigenvalues set the growth, decay, and oscillation of each mode, and thus stability, memory, and rhythm.

Next: probability again, now armed with vectors and matrices.